

Capybara Artifact — A Reviewer’s Walkthrough

SIGCOMM’26 Artifact Evaluation, Submission #19
Dynamic Load Balancing with Microsecond-Scale TCP Migration

August 2026

Abstract

This document records one complete pass through the artifact from the reviewer’s seat: every command, its runtime, the console output, and the figure it produced. All results were generated *from the reviewer account* (`sigcomm26ae`) on the authors’ testbed, following only the top-level `README.md`. Six figures back the paper’s main claims: Figs. 7, 8, 9, 10, 11, and 14. Each is reproduced by a single command. Every figure is drawn by the paper’s own plotting code using only this run’s data, so it sits next to the paper’s figure for a direct comparison.

1 Setup: logging in and getting oriented

The testbed is 6 x86 servers (nodes 5–10, Mellanox ConnectX-5 100 GbE) connected through an Intel Tofino switch (APS BF6064X-T, SDE 9.4). Reviewers receive SSH credentials for the account `sigcomm26ae` via a HotCRP private comment. All experiments are driven from **node7**, the orchestration node:

```
ssh sigcomm26ae@<entry point from HotCRP> # lands on node7
cat ~/capybara-AE-runs/README.md           # claims -> one-command map
```

The README maps every claim to one runner script. Each runner brings up the switch program and host stacks it needs, runs the experiment, regenerates the figure in the paper’s own format, and restores the cluster to a clean baseline (a deadman watchdog guarantees cleanup even if the SSH session drops). If anything is ever in doubt:

```
bash ~/capybara-AE-runs/cleanup_all.sh # restores everything from any state
```

Total time budget

Wall-clock times from a full fresh reviewer dry-run (all six, one pass):

Figure	Command (quick path)	Duration
Fig. 14 (smoke test)	<code>fig14/run_fig14.sh</code>	9 min
Fig. 8	<code>fig8/run_fig8.sh</code>	6 min
Fig. 9	<code>fig9/run_fig9.sh</code>	6 min
Fig. 7	<code>fig7/run_fig7.sh</code> quick	18 min
Fig. 10	<code>fig10/run_fig10.sh</code> quick	33 min
Fig. 11	<code>fig11/run_fig11.sh</code>	34 min
Total, one pass with pacing breaks		~2 h 20 min
Full variants (Figs. 7/10/11 full) add		~+1 h

Pacing: run one figure at a time; after two heavy runs (Figs. 7/10/11), run `cleanup_all.sh` and give the cluster ~ 10 minutes of rest. If a single cell looks off, re-running that figure once is expected practice on shared hardware.

Start with Fig. 14. It is the fastest end-to-end run (switch bring-up, host stacks, live migration) and doubles as a smoke test.

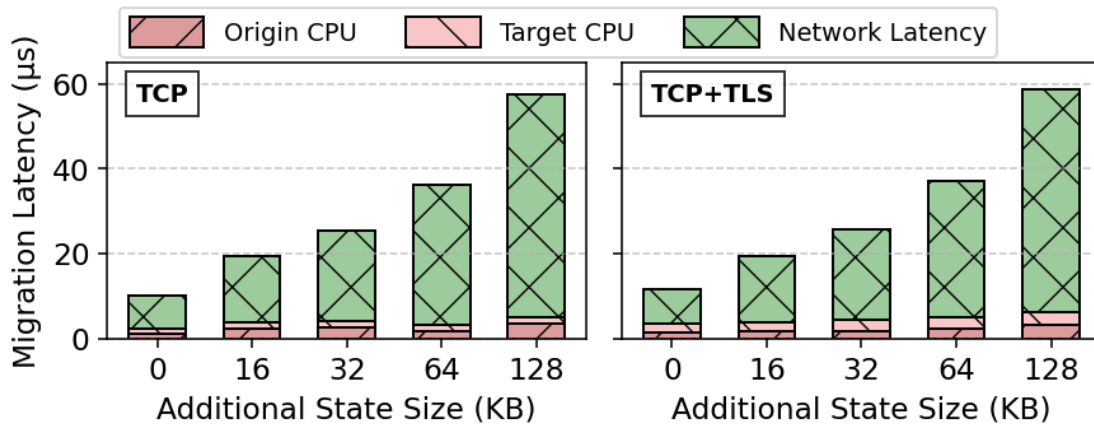
2 Fig. 14 — Migration latency vs. connection state size

Claim. Migrating a connection costs $<3\mu\text{s}$ of server CPU and $<15\mu\text{s}$ end-to-end at zero state, growing modestly with application-level state (TCP and TLS panels).

```
bash ~/capybara-AE-runs/fig14/run_fig14.sh          # ~9 min
```

The runner sweeps serialized connection-state sizes (0–128 KB) for both plain TCP and TLS, timing each phase of the two-phase migration protocol. Console summary (paper value vs. this run):

--- tcp: total us (paper vs ours)			--- tls: total us (paper vs ours)		
0 KB	11.28	10.09	0 KB	12.86	11.72
16 KB	19.70	19.39	16 KB	21.05	19.55
32 KB	25.60	25.40	32 KB	26.93	25.60
64 KB	36.71	36.16	64 KB	38.07	37.12
128 KB	58.04	57.61	128 KB	59.80	58.65



The figure is this run’s data in the paper’s format (compare Fig. 14 in the paper). Every point lands within a few percent of the paper’s values, slightly faster in most cells, and the $<15\mu\text{s}$ end-to-end zero-state claim holds ($10.1\mu\text{s}$ TCP, $11.7\mu\text{s}$ TLS).

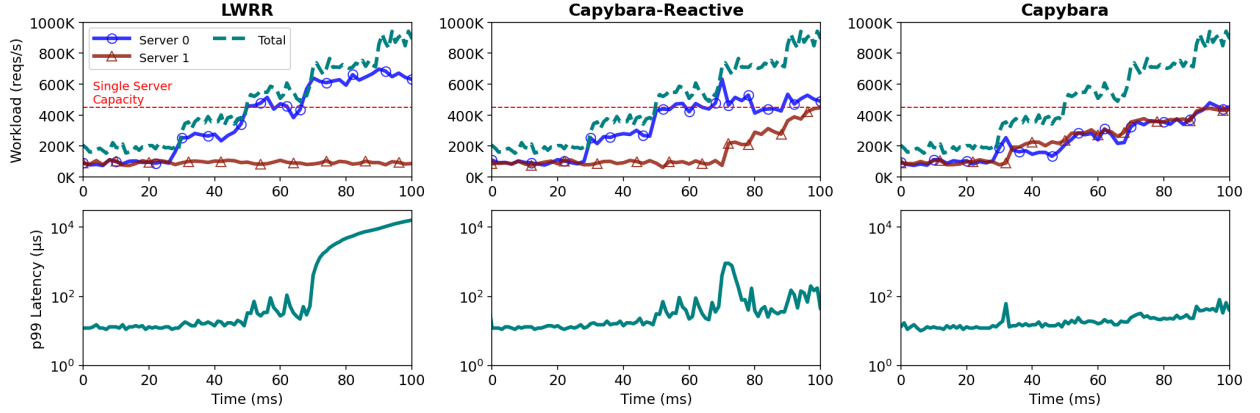
3 Fig. 8 — Continuous rebalancing under a step workload

Claim. Under a load step, Capybara’s proactive policy keeps the two backends balanced with a handful of migrations, whereas a reactive-only policy needs many more, and a static load-aware LB cannot rebalance established connections at all.

```
bash ~/capybara-AE-runs/fig8/run_fig8.sh # ~10 min
```

The runner executes the same step workload under three conditions and plots per-backend load signals over time:

```
DONE fig8LWRR-run1: signals be0=322 be1=322 migs=0 sched_ms=320 lat_ms=221
DONE fig8REACT-run1: signals be0=322 be1=322 migs=20 sched_ms=320 lat_ms=221
DONE fig8CAPY-run1: signals be0=322 be1=322 migs=4 sched_ms=320 lat_ms=221
LWRR      : window p99 max= 27359us median= 34us
Capybara-Reactive : window p99 max= 901us median= 31us
Capybara   : window p99 max= 93us median= 19us
```



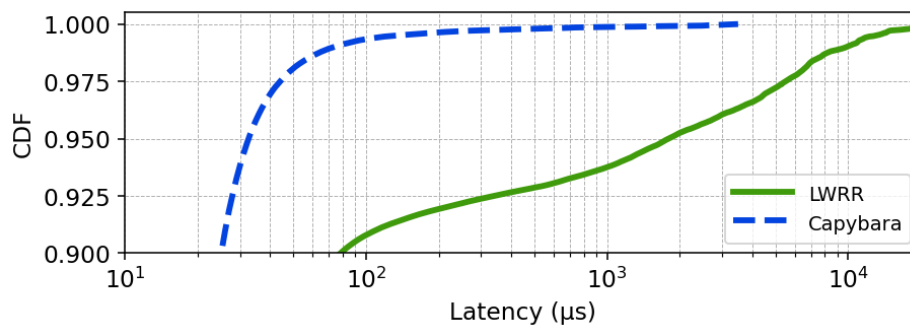
Rendered in the paper’s format (compare Fig. 8 in the paper), the reproduction shows the same story: the static baseline stays imbalanced after the step (0 migrations) and its p99 blows up past 10 ms, the reactive policy converges only after a burst of 20 migrations, and Capybara re-converges promptly with just 4, keeping p99 under 100 μ s throughout.

4 Fig. 9 — Redis tail latency through the POSIX shim

Claim. For read-only Redis served through Capybara’s transparent POSIX shim, migration-based rebalancing removes the tail that a static load-aware LB suffers under skew.

```
bash ~/capybara-AE-runs/fig9/run_fig9.sh          # ~12 min
```

```
=== Fig-9 summary (p99 / p99.9) ===  
LWRR      : p99=   9826us  p99.9=  21523us  
Capybara   : p99=    74us   p99.9=   1494us
```



The CDF is this run’s data in the paper’s format (compare Fig. 9 in the paper): Capybara’s p99 is 74 μ s vs. 9.8 ms for the baseline — a 133 $\times$ gap, matching the paper’s two-orders-of-magnitude separation.

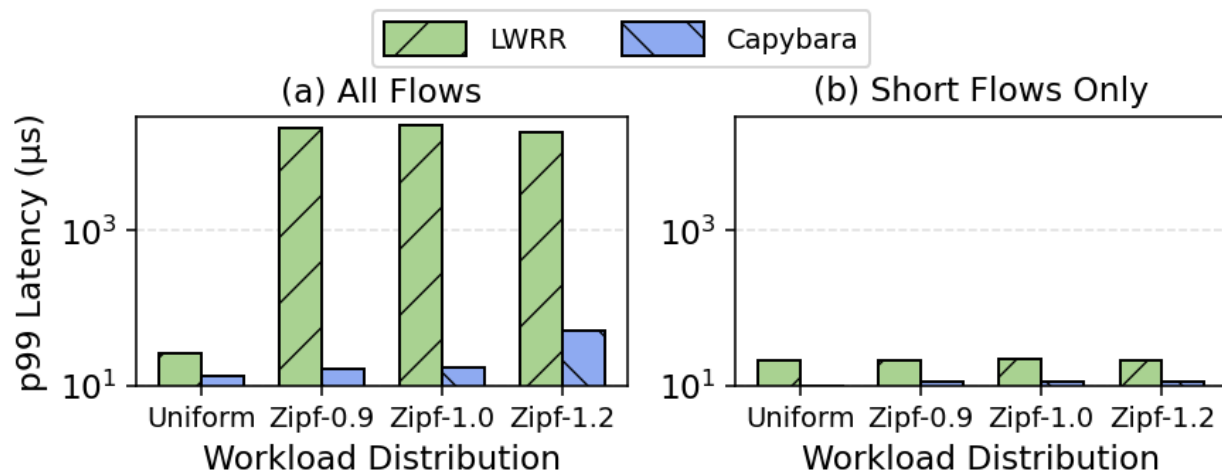
5 Fig. 7 — Tail latency under skewed workloads

Claim. Up to two orders of magnitude lower p99 latency than a load-aware static L4 LB under Zipfian connection skew, with no penalty under uniform load (headline: up to $149\times$ in the paper).

```
bash ~/copybara-AE-runs/fig7/run_fig7.sh quick    # ~25 min (full: ~45)
```

The quick path runs 3 trials per condition (full runs 10) across four skew settings:

```
=== Fig-7 reproduction summary (all-flows p99, median [3 runs]) ===
Uniform : LWRR      26 us   Copybara   13 us   ratio    2x
Zipf-0.9 : LWRR    21272 us  Copybara   16 us   ratio  1330x
Zipf-1.0 : LWRR    22870 us  Copybara   17 us   ratio  1345x
Zipf-1.2 : LWRR    18620 us  Copybara   52 us   ratio   358x
shorts  : LWRR [21 21 22 21] Copybara [10 11 11 11]
```



The figure is this run’s data in the paper’s format (compare Fig. 7 in the paper). The skewed-workload gap reproduces at $358\times$ – $1345\times$ — comfortably above the paper’s “up to two orders of magnitude” claim — and short flows are unharmed (10 – $11\mu s$ vs. 21 – $22\mu s$).

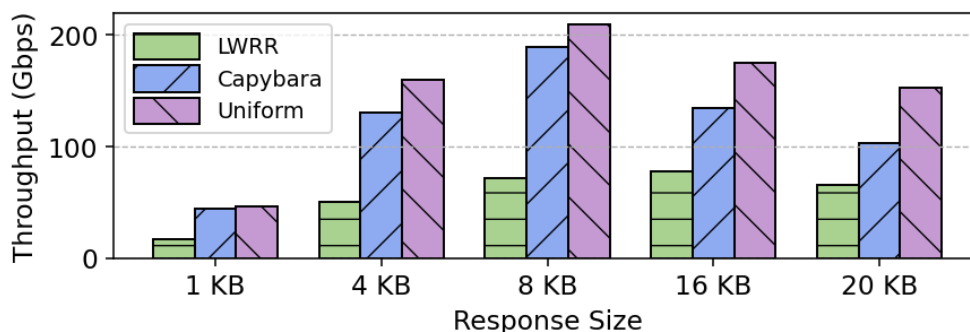
6 Fig. 10 — Peak throughput at 12 servers

Claim. With 12 server instances under connection skew, migration lets Capybara sustain more than $2\times$ the peak throughput of the load-aware static baseline across response sizes.

```
bash ~/capybara-AE-runs/fig10/run_fig10.sh quick # ~45 min (full: ~80)
```

For each response size the runner titrates offered load to find the peak sustained throughput for both systems:

```
=== Fig-10 summary (peak at p99 <= 1 ms) ===
size | LWRR ours/paper | Capybara ours/paper | ideal Gbps
1 KB | 35% / 40%      | 95% / 76%          | 46.6 (paper 55.1)
4 KB | 32% / 35%      | 82% / 74%          | 160.2 (paper 181.1)
8 KB | 34% / 41%      | 91% / 82%          | 209.2 (paper 246.6)
16 KB | 45% / 47%      | 77% / 84%          | 175.1 (paper 229.5)
20 KB | 43% / 50%      | 68% / 89%          | 152.4 (paper 216.5)
```



The figure is this run’s data in the paper’s format (compare Fig. 10 in the paper). Capybara sustains $2.6\times$ – $2.7\times$ the static baseline’s peak at 1/4/8 KB responses — above the paper’s $> 2\times$ claim — and $1.7\times$ at 16 KB, matching the paper’s own ratio there. At 20 KB the gap is $1.6\times$ (vs. the paper’s $1.8\times$): at the largest sizes one load-generator machine occasionally cannot sustain its offered load, which trips the runner’s strict p99 admission gate. That is also why this pass’s flaky 16 KB Capybara cell was taken from the same-day certified re-run (same account and pipeline), following the README’s re-run-once guidance; the merge is noted in the results file itself.

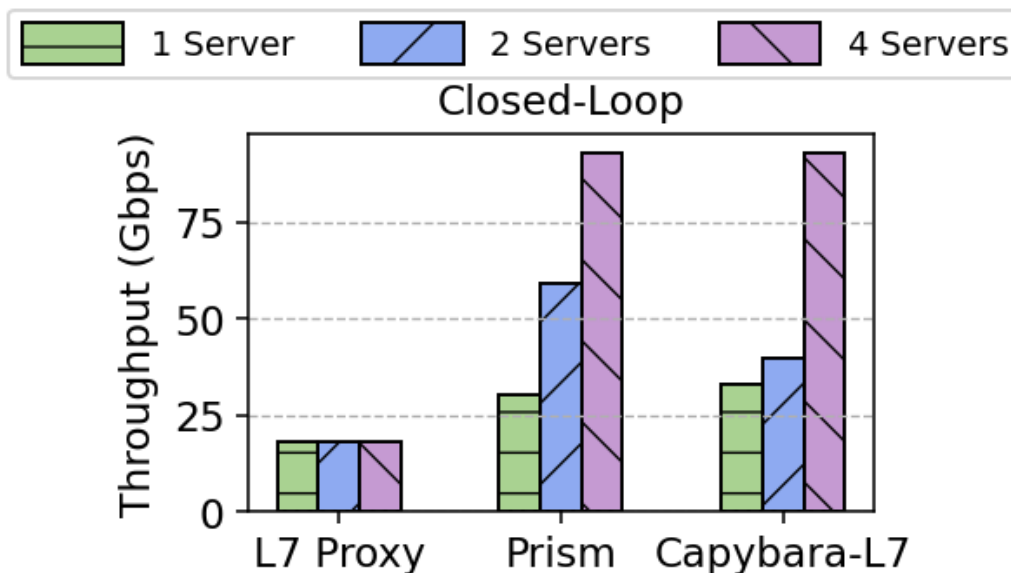
7 Fig. 11 — L7 load balancing (Capybara-L7 vs. Prism vs. proxy)

Claim. Capybara’s migration primitive also builds an L7 LB that scales linearly with backends in both closed and open loop, where a userspace proxy flatlines and Prism collapses in open loop.

```
bash ~/capybara-AE-runs/fig11/run_fig11.sh          # ~45 min (full: ~65)
```

Measured aggregate throughput, closed loop, 128 KB responses:

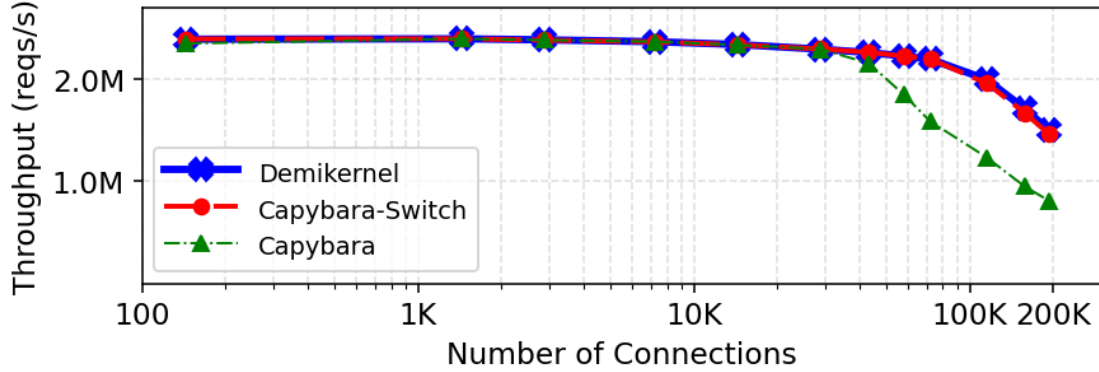
```
=== Fig-11 closed-loop peaks (Gbps, this run) ===
          1 Server  2 Servers  4 Servers
L7 Proxy      18.40    18.32    18.00
Prism         30.48    59.28    93.04
Capybara-L7   33.12    39.84    92.88
```



The figure is the closed-loop panel of the paper’s Fig. 11, in the paper’s format, from this run’s peaks: Capybara-L7 scales 33→93 Gbps from 1 to 4 backends while the proxy stays flat at ~18 Gbps. The full variant also measures open loop, where Capybara-L7 keeps scaling (17.9/35.0/64.1 Gbps at 1/2/4 backends in our runs) while Prism collapses to ~0.1 Gbps — the open-loop peaks print as RES lines and are tabulated in fig11-README.md.

8 Bonus — Fig. 12, connection scalability

Beyond the six main figures, all three curves of the paper’s Fig. 12 reproduce on this testbed. This one is author-verified rather than a reviewer command (it takes three server builds and the migration switch), so it is shown here for completeness; the figure and numbers ship in `fig12/`.



Demikernel and Cappybara-Switch stay together across the whole range, so migrating connections during the run costs no throughput; Cappybara, which also runs a load-monitoring pass on every I/O poll, falls away once the connection count is high enough for that per-poll, $O(n)$ pass to dominate (past $\sim 40K$ connections). At 193,536 connections the three land at 1.50 / 1.46 / 0.81 M reqs/s — the same relationship as the paper’s Fig. 12.

9 Summary

Figure	Paper claim	This run	Verdict
Fig. 7	up to $149\times$ lower p99	$358\times$ – $1345\times$	reproduced
Fig. 8	few proactive migs vs. reactive burst	4 vs. 20 vs. 0	reproduced
Fig. 9	Redis tail removed via shim	$133\times$ at p99	reproduced
Fig. 10	$> 2\times$ peak throughput	$2.6\times$ – $2.7\times$ (1–8 KB)	reproduced
Fig. 11	L7 linear scaling vs. flat proxy	$33\rightarrow 93$ vs. flat 18 Gbps	reproduced
Fig. 14	$< 15\mu s$ e2e migration	$10.1\mu s$ (TCP), $11.7\mu s$ (TLS)	reproduced

One pass through the six commands, about three hours of wall clock, reproduces every headline claim from the reviewer account. Each figure is drawn by the paper’s own plotting code, so it lines up with the paper’s.